

AutonoTrust — Full Project Explainer (Basics → Advanced → Project Level)

PART 1: THE ABSOLUTE BASICS (so nothing is assumed)

What is "Agentic AI," really?

A normal chatbot (like plain ChatGPT) does one thing: you ask, it answers. It has no memory of what it did five minutes ago unless you paste it back in, it can't take actions in the real world, and it can't decide on its own what to do next.

Agentic AI means giving an AI:

1. **A goal**, not just a question ("get this task done" instead of "answer this question")
2. **Tools** it can use (search the web, run code, call an API)
3. **The ability to plan** — break the goal into steps itself
4. **Memory** — remember things across steps, or even across separate runs
5. **The ability to check its own work** and try again if it's wrong

An "agent" is one AI doing this. A **multi-agent system** is several AIs, each with a different job, working together — like a small team instead of one person doing everything.

What is Web3 / blockchain / smart contract, really?

Forget the hype. Here's the actual mechanic:

- A **blockchain** is a shared record book (a database) that no single person owns or controls. Once something is written to it, nobody — not even the people who built it — can secretly change it.
- A **smart contract** is a small program that lives on this shared record book. It's not owned by any one person. It runs exactly as written, automatically, whenever someone calls it. Nobody can pause it, edit it, or override it (unless the code itself allows that).
- A **wallet** is just a pair of keys (like a username + password, but cryptographic). One wallet = one identity that can hold money or sign transactions on the blockchain.
- A **transaction** is any action recorded on the blockchain — usually "send money from wallet A to wallet B" or "call this function in this smart contract."
- **Testnet** = a practice version of the blockchain. Same technology, but the "money" (test ETH) is fake and free, given out by "faucets." Nothing here has real-world value. This is where you build and demo — never real/"mainnet" money for a course project.

The one thing blockchain is actually good for, that normal software isn't: normal software is controlled by whoever runs the server. If I write a Python script that says "if task passed, send money," *I* am the one deciding whether that's true — you have to trust me. A

smart contract removes that requirement: the *rule* is public and unchangeable, so nobody has to trust a specific person, only the code, which everyone can inspect.

PART 2: THE PROBLEM STATEMENT — BUILT UP FROM SCRATCH

Step 1: The everyday version of the problem

Imagine you hire a freelancer online to write some code for you. Two things could go wrong:

- You pay first → they take the money and vanish, or deliver garbage
- They work first, you pay after → you could refuse to pay even if they did great work

This is called the **trust problem**, and it's why platforms like Upwork/Fiverr exist — they act as a trusted middleman holding the money until both sides are happy.

Step 2: Why this gets worse with AI agents

Now imagine that instead of a human freelancer, it's an **AI agent** doing the work — and it's hiring/being hired by *other* AI agents, with no human directly supervising every step (this is genuinely happening right now — it's called the "agentic economy," where AI agents act as independent workers, negotiators, and even customers).

An AI agent has:

- No bank account a court can freeze
- No reputation system that follows it around
- No legal accountability
- No inherent reason to be honest — it does whatever it's told or whatever maximizes its objective

If AI agents are going to autonomously hire each other, delegate work, and pay for completed work — **who do you trust to hold the money and decide who's right?** A single company's server? That's just moving the "trust a middleman" problem onto a different middleman, and it can be hacked, biased, or simply lie.

Step 3: The precise problem statement

How can autonomous AI agents complete real work for each other and get paid — with a guarantee that payment only happens if the work is verifiably good — without needing a trusted human or company to manually check and approve every single transaction?

That's the actual, specific problem. Not "let's use AI," not "let's use blockchain" — this specific trust gap in autonomous multi-agent collaboration.

PART 3: THE SOLUTION — WALKED THROUGH STEP BY STEP

The one-sentence solution

A team of specialized AI agents plans, does, and independently checks a piece of work — and a smart contract holds the payment in escrow, releasing it **only** when the independent checking agent (cryptographically, provably) confirms the work is good — with the full history of who did what, and how reliable each agent has been, permanently recorded where nobody can fake it.

Now let's walk through exactly what happens, in order

- 1. A task comes in.** Someone (a simulated "client") describes what they want done — e.g., "write a Python function that calculates the nth Fibonacci number."
- 2. The Requester Agent turns this into a real contract.** Vague requests are impossible to judge fairly. This agent rewrites the request into a strict, structured spec: exact deliverables, and — critically — **objective, testable acceptance criteria** (e.g., "must pass these 5 unit tests"), not vague ones like "should be good code." *Concept demonstrated: Planning.*
- 3. The Matcher Agent picks who does the work.** It looks at a pool of "Executor" agents, each with different skills (one is good at code, one at research, etc.), and checks their **on-chain track record** — how often has each one succeeded before? Specifically, it uses RAG to retrieve the top-3 most similar *past tasks* (by semantic similarity to the current task's description) and checks which Executor(s) succeeded on those — then combines that with each candidate's on-chain reputation score to make the final pick. An LLM only steps in to break ties between close candidates. *Concept demonstrated: Memory (the system remembers and uses history) + retrieval (semantic similarity, not just a raw counter).*
- 4. The Client's payment gets locked in a smart contract (this is Transaction #1).** Before any work starts, the agreed payment amount (in free testnet currency) is sent to the smart contract, not to the Executor directly. The money is now stuck there — nobody can access it — until the contract's rule is satisfied.
- 5. The Executor Agent does the actual work.** It uses real tools — a code interpreter, web search, or document retrieval, depending on the task type. **For research/writing tasks**, it uses **RAG** (Retrieval-Augmented Generation) to pull facts from real source documents in the Chroma vector store instead of making things up — so its output can later be checked against something real. **For the coding demo task specifically (e.g., the Fibonacci example), RAG is not used at all** — there are no source documents to retrieve from; the Executor just writes code directly and the Auditor verifies it by running tests, not by checking retrieval-grounded claims. This is worth stating plainly in your report so it's clear RAG is applied only where it's actually useful, not everywhere. *Concept demonstrated: Tool use, and (if it reasons → acts → observes → repeats) the ReAct pattern.*

Safety note on the Executor's code tool: letting an AI agent run arbitrary code is genuinely risky if done carelessly (it could run something destructive or resource-hungry). The Executor's code tool should run in a **sandboxed subprocess** with strict resource limits (memory cap, CPU time limit) and a hard timeout, and no access to the host filesystem or network beyond what's explicitly allowed. This isn't optional polish — it's a basic safety requirement any grader would expect you to have thought about.

6. The Auditor Agent independently checks the work. For a coding task, this is beautifully objective: it actually **runs the hidden unit tests** and sees if they pass — no opinions, just facts. For research/writing tasks, it uses RAG too — it retrieves the same source material the Executor used and checks the claims against it, plus looks up how similar past tasks were judged, to stay consistent. *Concept demonstrated: Reflection + agentic RAG.*

7. If it fails, it doesn't just die — it retries with feedback. The Auditor's specific critique goes back to the Executor, which tries again (up to a set number of retries). Every attempt and verdict is kept in a growing history, not thrown away. *Concept demonstrated: Reflexion (the "try again, but smarter" loop).*

8. Only the Auditor's own wallet can trigger payment (Transaction #2). This is the most important trust detail: the smart contract is written so that **only the specific wallet belonging to the assigned Auditor** is allowed to call the "release payment" function. Even if your backend server were hacked, or you personally tried to cheat, the contract would refuse — it only listens to the Auditor's cryptographic signature. If it passed: money goes to the Executor. If it failed after all retries: money goes back to the Client automatically.

9. Reputation gets updated on-chain (can be the same transaction as #2). The Executor's (and Auditor's) success record is updated in the smart contract — permanently and publicly. This is what step 3 reads from next time, so good agents get picked more often, automatically, without anyone manually managing a leaderboard.

10. Safety nets, in case something breaks.

- If the Executor never finishes in time, the Client can reclaim their money after a deadline (Transaction, if needed)
- If retries run out, automatic refund — no funds get stuck forever

11. A Reporter Agent writes the final summary — what happened, what was checked, the transaction hash (a link the Client can click to see the real, verifiable transaction on a blockchain explorer), and the outcome.

PART 4: ANSWERING EVERY "WHY" QUESTION DIRECTLY

Q: Where exactly are the blockchain transactions, and who makes them? There are 2-4 real transactions per completed task:

1. Client locks payment into the contract (before work starts)

2. Auditor submits its verdict, which the contract uses to release or refund funds
3. (Optional/combined) Reputation gets updated
4. (Only if something goes wrong) Client reclaims funds after a timeout

None of these are made by "the AI agent" in some magical sense — an LLM can't hold a private key or physically sign anything. What happens is: the AI agent produces a *decision* (text/JSON), and your backend code, using the wallet **belonging to that specific agent role** (Auditor's wallet, Client's wallet, etc.), submits the transaction that carries out that decision. The AI reasons; the wallet executes.

Q: Can agents "give money" once a task is done? How? Not directly and not unilaterally — and that's intentional, not a limitation. No single agent (including the Auditor) can just "send money." What the Auditor's wallet *can* do is call one specific, narrow function on the smart contract (`submitVerdict`). The contract itself — code that nobody can secretly alter — is what actually decides to move the funds, and only because its encoded rule was satisfied. This is the entire point: **no individual party, human or AI, has the power to move the money alone.** Only the agreed-upon, transparent rule does.

Q: Why not just use a normal database instead of blockchain? Because a database is controlled by whoever runs the server — meaning *you*, the developer, could quietly edit "failed" to "passed" and nobody would know. With the smart contract, that's not possible: the rule (only the Auditor's wallet can trigger release, and only if it says "passed") is enforced by code everyone can inspect, not by trusting a person. If you removed the blockchain from this project, you'd lose the entire trust guarantee — that's the test for whether it's genuinely needed here, and it passes.

Q: Why RAG, specifically — where does it actually help, not just exist for the sake of the syllabus? Two places, both load-bearing:

- The **Executor** (for research/writing tasks) needs to ground its output in real source documents — otherwise it's just making things up, and there'd be nothing objective for the Auditor to check.
- The **Auditor** retrieves similar past verified/rejected tasks before judging the current one, so its judgment is based on precedent, not a fresh, inconsistent opinion every time. If you removed RAG from these two spots, the Executor could hallucinate freely and the Auditor's judgments would be inconsistent — so both uses genuinely solve a real weakness, not decoration.

Q: Why do you need multiple agents instead of one big AI doing everything? Because if the same AI does the work *and* checks its own work, it has no reason to catch its own mistakes — that's not real verification, it's just the same biased judge grading its own exam. Separating "do the work" (Executor) from "check the work" (Auditor), with different tools and different wallets, is what makes the verification meaningful — structurally, not just by prompt-writing.

Q: Is this cost-effective? Will it increase costs? No — everything runs on free tiers, by design:

- Testnet blockchain = free fake money from a faucet, forever, no real value ever at risk
- RPC calls (reading/writing to the blockchain) = free tier from Alchemy/Infura, far more capacity than a course demo needs
- LLM calls = free tier from Groq (fast, free, rate-limited — mitigated by having a backup free provider ready)
- Vector database (for RAG) = Chroma, runs locally, \$0
- Hosting = Render/Vercel free tiers The only real risk isn't cost, it's rate limits during a live demo (too many requests too fast on the free LLM tier) — solved by having a second free provider as backup, not by spending money.

Q: Is it safe to let an AI agent run arbitrary code? Not by default, no — an LLM-generated script could accidentally (or, if prompted adversarially, deliberately) do something harmful: infinite loops, huge memory usage, filesystem access, network calls. The fix is standard sandboxing: the Executor's code tool runs the generated code in an **isolated subprocess** with a memory cap, a CPU/wall-clock timeout, and no filesystem or network access beyond what's explicitly needed. The Auditor then runs the hidden unit tests against that same sandboxed output. This is a basic, expected safety measure — worth one sentence in your report so it's clear you considered it rather than blindly executing LLM output.

Q: What happens if the Executor never finishes, or the Auditor goes offline? The contract has a deadline. If it passes with no verdict submitted, the Client can call a "reclaim" function and get their money back automatically — funds are never permanently stuck.

Q: Isn't the "client," "money," and "task" all simulated? Yes, and that's normal and fine to state clearly in your report: the *task budget* is a made-up number (testnet currency has no real value), but the *mechanics* — transactions, wallets, smart contract enforcement, on-chain reputation — are all real and verifiable on a testnet block explorer. This is exactly how legitimate Web3 hackathon/course projects are built and demoed, since nobody should be risking real money in a classroom setting.

PART 5: WHY BOTH AGENTIC AI AND WEB3 — NOT EITHER ALONE

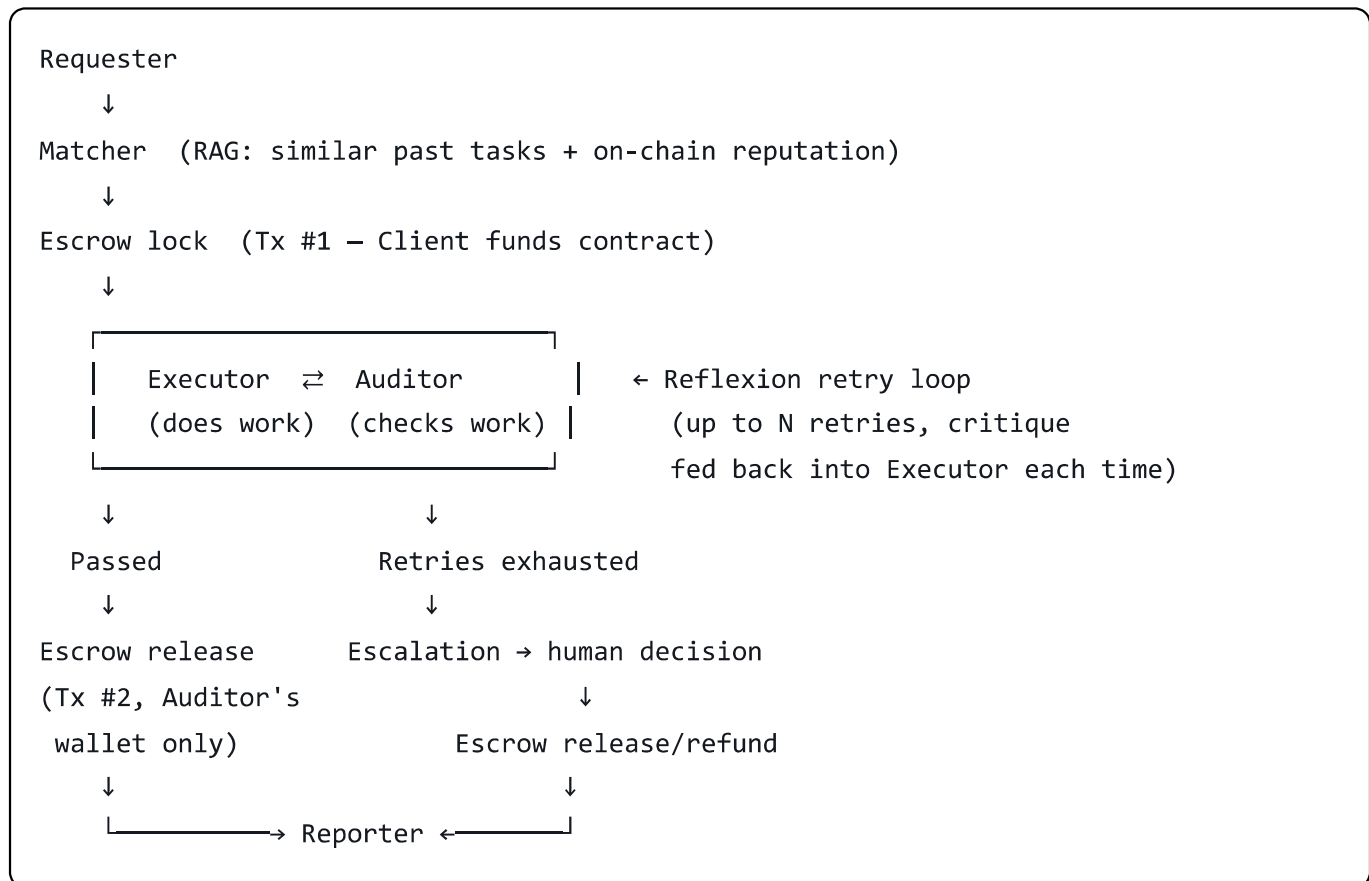
- **Agentic AI alone** (no blockchain): you'd still need a trusted human to manually check the Auditor's work and manually release payment — because nothing stops a compromised or dishonest process from lying about the outcome. Not truly autonomous.
- **Web3 alone** (no AI): a smart contract can hold money and enforce rules, but it has zero intelligence — it can't plan a task, do research, write code, or judge whether work is actually *good*. It only knows how to follow a yes/no instruction.

- **Together:** the AI provides the intelligence (planning, doing, judging), and the blockchain provides the enforcement (making sure the AI's judgment is honestly acted upon, without anyone able to interfere). Each solves exactly the part the other can't.

PART 6: WHAT YOU ARE ACTUALLY BUILDING (project-level summary)

Agents (LangGraph nodes): Requester → Matcher → Executor (code / research / writing, tool-specific) → Auditor (with retry/Reflexion loop) → Escrow release → (Escalation if needed) → Reporter

State machine, visually:



This is the diagram worth putting on a slide — it makes the Reflexion loop and the Auditor-trigger visually obvious at a glance.

Web3 layer: One Solidity smart contract on a testnet (Base Sepolia or Polygon Amoy) with functions for locking payment, submitting a verdict (restricted to the Auditor's own wallet), timing out, and tracking reputation.

RAG layer: One shared Chroma vector store, used by the Executor (grounding in source documents) and the Auditor (grounding in past precedent) — not sprinkled everywhere, only where retrieval genuinely improves a decision.

Demo centerpiece: A concrete, objectively-checkable coding task (e.g., "write `fib(n)`, pass 5 hidden unit tests") so the Auditor's verdict is a verifiable fact, and the final proof is a real,

clickable transaction hash on a testnet block explorer — the moment that makes the whole system feel real rather than simulated.

What this demonstrates, mapped directly to what agentic AI courses grade: planning, tool use, memory, ReAct-style reasoning, Reflexion (retry-with-feedback), multi-agent orchestration with genuinely distinct roles, agentic RAG, and human-in-the-loop escalation — plus a Web3 trust layer that is structurally necessary to the core problem, not decorative.